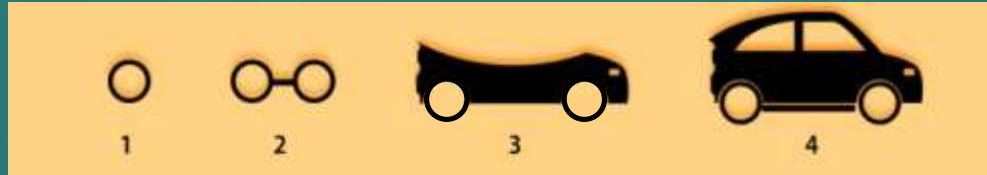


- i. Which one illustrates the **breadth-first iterative** approach?
(send answer as a **private** Zoom message, to Damith)

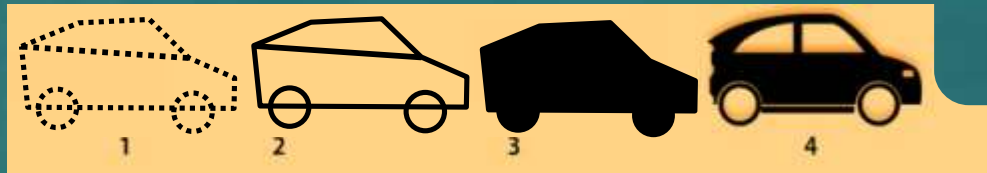
(a)



(b)



(c)



iv) Which is the best for software projects?

- ii. Which is likely to be used when a car manufacturer builds a new type of a car?
iii. Which would you use if *you* were asked to build a new type of a car?
In software projects, (b) is possible and sometimes necessary, because ...

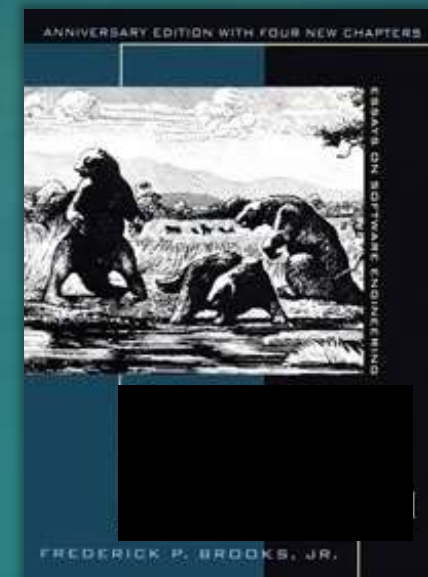
Finally, there is the delight of working in such a tractable medium. The programmer, like the poet, works only slightly removed from pure thought-stuff. He builds his castles in the air, from air, creating by the exertion of the imagination. Few media of creation are so flexible, so easy to polish and rework, so readily capable of realizing grand conceptual structures....

Fourth is the joy of always learning, which springs from the nonrepeating nature of the task. In one way or another the **problem is ever new**, and its solver learns something: sometimes practical, sometimes theoretical, and sometimes both.

Instructions for this lecture:

- **Ensure your Zoom name matches** the one in your student card.
(for **participation tracking**)
- **When answering a question I asked**, send as a **private message** to me.
- **When asking a question from me**, use Zoom **Q&A** feature (not the normal chat).
- **For urgent issues** unmute and speak e.g, if I seem to be pointing at something on the slide but you don't see my **laser pointer on screen**

Seen these before?



Example adapted from a presentation by Spotify

In our tutorials (even in F2F ones), when the tutor asks a question, all students use Zoom private messages to submit the answer. This way, everyone gets to think and answer questions without being influenced by others' answers.

Next week's tutorial will be a mass tutorial via Zoom (due to holidays) i.e., 400+ students sending private messages to one person. We wish to do a dry run today, to see if Zoom can scale up to a such high volume of private messages.

So, I will ask you questions at times (like I did in the previous slide), and you should send your answer as a private message too. We'll count these towards bonus participation marks.

Pretend you are the only student in the class, and I'm taking directly to you 😊

Instructions for this lecture:

- Ensure your Zoom name matches the one in your student card. (for participation tracking)
- When answering a question I asked, send as a private message to me.
- When asking a question from me, use Zoom Q&A feature (not the normal chat).
- For urgent issues unmute and speak e.g, if I seem to be pointing at something on the slide but you don't see my laser pointer on screen

Weekly ~~lecture~~ *briefing* structure

Recap of the last week 

+

Preview of the next week

Course Briefing Videos from last weeks 1, 2...

[MUST-WATCH] Briefing Videos for Week 1

Before you do anything else please watch the following briefing videos in the order given. NUSNET login is required to access videos.

Watching course briefing videos counts for participation.

When watching a briefing video for the first time, avoid scrubbing or jumping ahead/around in the video timeline, as this can interfere with the video platform's capturing of what part of the video you actually watched. However, watching at faster speeds is fine.

 **We have added closed captioning support** for course briefing videos.

1. Welcome to the Course:

>  **Video** 1 min

2. Course Goals: How the goals of this course shape its structure (possibly different from other courses), and how it should be approached by students.

>  **Video** 9 mins

3. Week 1 Briefing: As this is the first weekly briefing, **it also covers some general points relevant to all future weeks.**

>  **Video** 4 mins

[MUST-WATCH] Briefing Videos for Week 2

This week's briefing videos start with a detour.

0. Detour: SDLC Basics: Watch the following video covering 'Basics', as the concepts covered in there are referenced in the

>  **Video** 6 mins -- Detour: **SDLC Basics**

1. Iterative + Brownfield = ???: How the iterative and brownfield approaches shape your learning experience.

>  **Video** 6 mins

2. Challenges and Solutions: How to tackle three challenges you'll face: *overload, workload, and technical problems.*

>  **Video** 11 mins

3. Week 1-2 Briefing (Week 1 Recap + Week 2 Preview): This video covers what you need to do in Week 2.

Note: While weekly briefings are normally done in hybrid mode, this is a recorded video, to allow you to get started with week 2 tasks early.

>  **Video** 4 mins

Course Briefing Videos from last weeks 1, 2...

Course Goals

Goal: Improve SE career prospects

Objective: Prepare you for internships

- ✓ Push: Strengthen self-survival skills
(e.g., self-learning, troubleshooting)
- ✓ Push: Immunize against mundane aspects of SE
- ✓ Push: Raise the baseline competency level

Course Briefing Videos from last weeks 1, 2...

Points to note about the weekly schedule

1. **No lectures; weekly briefings only** – mostly optional.
2. After week N briefing (on week N Friday),
you can to move to week N+1 early!
3. **Weekly schedule page has all** info you need
for that week.
4. **Self-study weekly topics.**
Submit weekly Canvas quiz to show your learning.
5. **Participation grading is forgiving.**
OK to miss a few items.



Course Briefing Videos from last weeks 1, 2...

Iterative + Brownfield = ✓✓✓

Sequential and iterative are two fundamental approaches to doing SE projects.
Sequential is easier for beginners, but **we must learn iterative** too (because of internships).

Greenfield is easier for beginners, but we **must learn to brownfield** too (because of internships).

We do a **greenfield iP** and a **brownfield tP**, both **breadth-first iterative**.

To help, we use:

1. iterative topic coverage
2. detailed instructions.
3. close monitoring

Side effects:

- jumping around topics
- a lot of detailed instructions to follow
- many small things to do every week

There are **countermeasures** built into the course to deal with them.

Course Briefing Videos from last weeks 1, 2...

Main challenges, and solutions

Challenge: Information overload

- ✓ You only need to read the weekly pages
- ✓ Make use of information layers
- ✓ Search, ask in the forum (don't ask tutors!)

Challenge: Workload

- ✓ No extra marks for exceeding effort requirements
- ✓ No extra marks for novel/unique/cool products
- ✓ tP workload to be cut down drastically

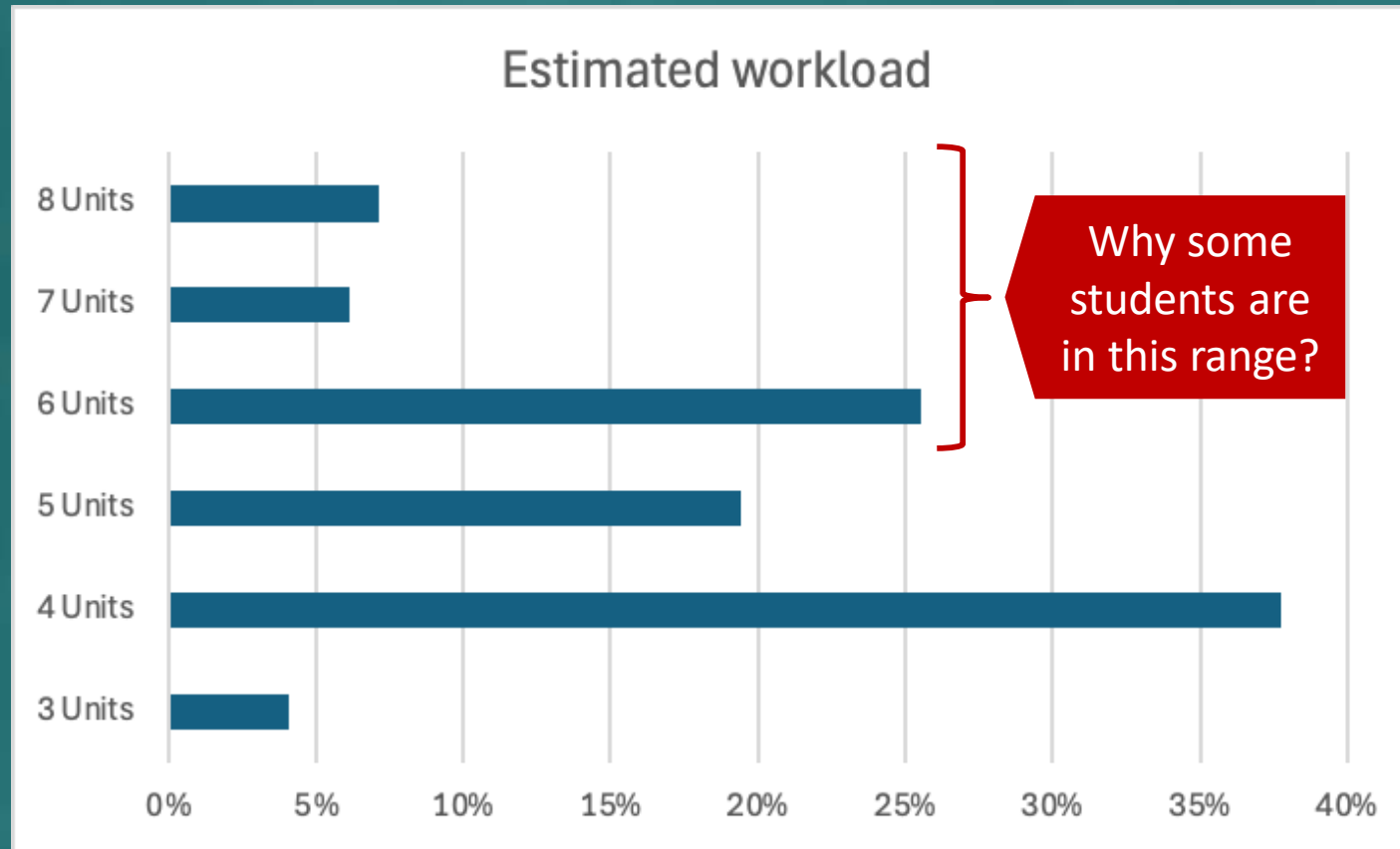
Challenge: Technical problems

- ✓ Use the forum (not ask tutors/instructors)



Anonymous post-course survey results

Compared to other 4-Unit courses, the workload of this course is equivalent to ...



Possible reason 1:
Doing more than needed.

Don't, unless you can afford it.

Possible reason 2:
Lack of programming skills

Good! You can get more out of this course (compared to strong programmers).

Possible reason 3:
Taking shortcuts to save time/effort and ending up wasting more time/effort.

Follow instructions, follow the schedule, and you'll be fine.

Go to Canvas → **Week 3 Quiz**

Which videos are compulsory to watch?

- ☐ W1 **briefing** videos labelled as MUST-WATCH
- ☐ W2 **briefing** videos labelled as MUST-WATCH
- ☐ Future **briefing** videos labelled as MUST-WATCH
- ☐ Zoom recording of this **weekly briefing**
- ☐ Zoom recording of future **weekly briefing**
- ☐ Videos given with textbook topics

About 'in-briefing questions':

Weekly Canvas quiz has a section for in-briefing questions.

These questions will be discussed during the briefing. You can enter the answer into Canvas during the briefing (or when watching the recording).

They are bonus questions. No penalty if you don't answer them.

Your answers are auto-saved. Don't click the 'Submit' button yet.

Later, answer the remaining questions (which are based on weekly topics), and submit the quiz.

Slides with such questions have a special sound effect, to alert you.

Any value in paying attention to weekly briefing videos?

Divided students into clusters based on their final marks (reason: to compare results among students of similar academic strength).

Removed students who did not answer in-lecture quizzes (reason: to remove the influence from quizzes).

In each cluster, we compared the average marks between these two categories:

- A. Students who attended the briefing live or watched the video
(i.e., paid attention to briefing videos)
- B. Students who simply skipped ahead in the video to find answers to in-lecture questions
(i.e., didn't pay attention to briefing videos)

Possible reason: weekly briefings revisit important parts of topics, which are also likely to be tested in the exam.

Typically, group A scored 1.5-2.5 marks higher than group B

Past week: recap

Topics:

- [W2.1] SE: Intro ⚡⚡⚡
- [W2.2] SDLC Process Models: Basics ⚡⚡
- [W2.3] RCS: Revision History ⚡
- [W2.4] RCS: Remote Repos ⚡
- [W2.5] IDEs: Basic Features ⚡
- [W2.6] Automated Testing of Text UIs ⚡⚡

Admin:

1. Submit post-lecture quiz ⏸ Fri. P
2. Get connected to our communication channels

iP:

1. Learn about the project
2. Set up prerequisites
3. Set up the project in your computer
4. Add Increments while committing frequently:
Level-1, Level-2, Level-3, Level-4, A-
TextUiTesting, Level-5, Level-6, A-Enums

Past week: recap

Topics:

- [W2.1] SE: Intro ⚡⚡⚡
- [W2.2] SDLC Process Models: Basics ⚡⚡
- [W2.3] RCS: Revision History ⚡
- [W2.4] RCS: Remote Repos ⚡
- [W2.5] IDEs: Basic Features ⚡
- [W2.6] Automated Testing of Text UIs ⚡⚡

Admin:

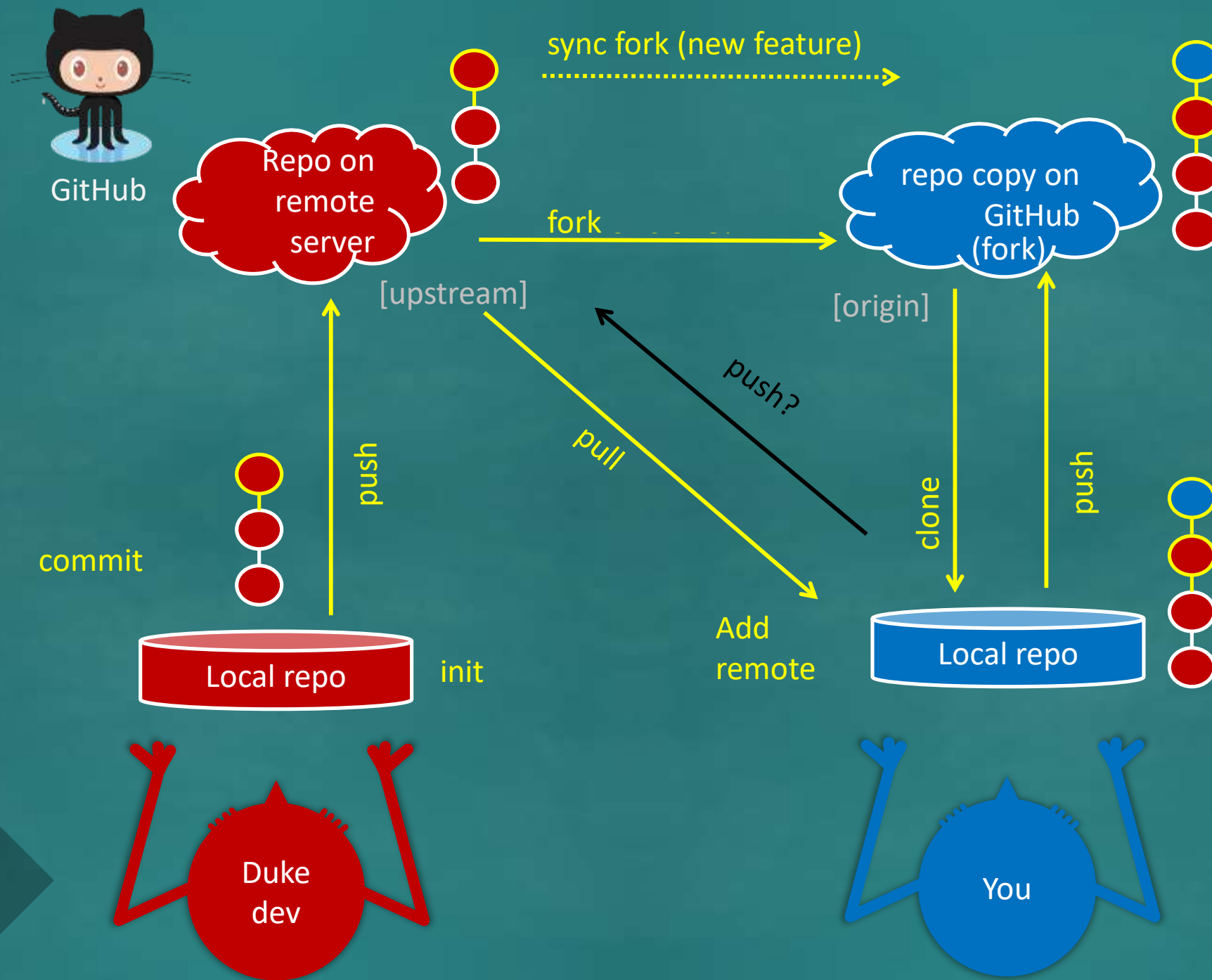
1. Submit post-lecture quiz ⏰ Fri. P
2. Get connected to our communication channels

iP:

1. Learn about the project
2. Set up prerequisites
3. Set up the project in your computer
4. Add Increments while committing frequently:

Level-1, Level-2, Level-3, Level-4, A-
TextUiTesting, Level-5, Level-6, A-Enums

- ✓ Init
- ✓ Fork
- ✓ Clone
- ✓ Commit
- ✓ Tag
- ✓ Pull
- ✓ Push



Past week: recap

Topics:

- [W2.1] SE: Intro ⚡⚡⚡
- [W2.2] SDLC Process Models: Basics ⚡⚡
- [W2.3] RCS: Revision History ⚡
- [W2.4] RCS: Remote Repos ⚡
- [W2.5] IDEs: Basic Features ⚡
- [W2.6] Automated Testing of Text UIs ⚡⚡

Admin:

1. Submit post-lecture quiz ⏸ Fri. 
2. Get connected to our communication channels

iP:

1. Learn about the project
2. Set up prerequisites
3. Set up the project in your computer
4. Add Increments while committing frequently:

Level-1, Level-2, Level-3, Level-4, A-
TextUiTesting, Level-5, Level-6, A-Enums



```
> git make a mess  
> git try to recover  
> git start over
```

ask AI,
ask in the forum,
...

If you worry that you'll
make things worse,
take a backup of the
folder first

This works because all
meta-data of the repo is stored
inside the repo folder itself.

Past week: recap

Topics:

- [W2.1] SE: Intro ⚡⚡⚡
- [W2.2] SDLC Process Models: Basics ⚡⚡
- [W2.3] RCS: Revision History ⚡
- [W2.4] RCS: Remote Repos ⚡
- [W2.5] IDEs: Basic Features ⚡
- [W2.6] Automated Testing of Text UIs ⚡⚡

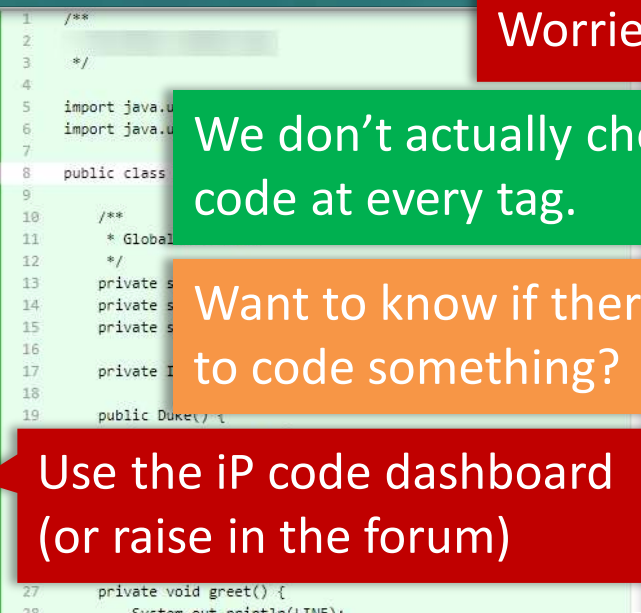
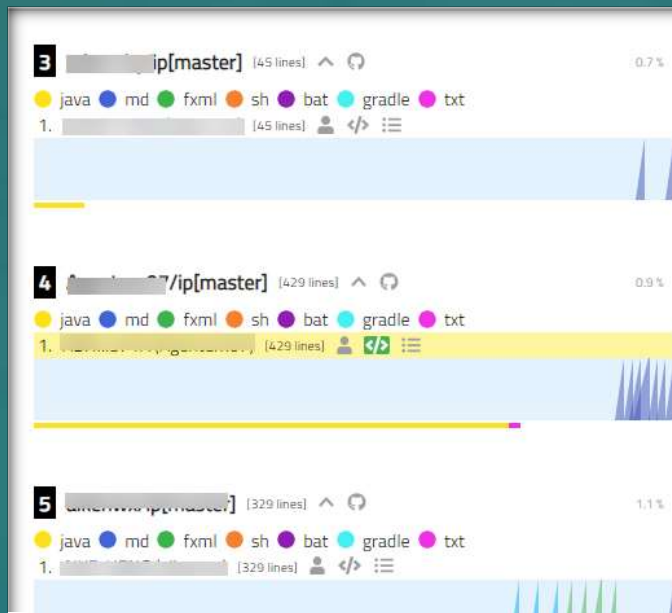
Admin:

1. Submit post-lecture quiz ⏰ Fri. 
2. Get connected to our communication channels

iP:

1. Learn about the project
2. Set up prerequisites
3. Set up the project in your computer
4. Add Increments while committing frequently:

Level-1, Level-2, Level-3, Level-4, A-
TextUITesting, Level-5, Level-6, A-Enums



Worried about grading?

We don't actually check/grade your code at every tag.

Want to know if there are others ways to code something?

Use the iP code dashboard (or raise in the forum)

Or ask ChatGPT

Some iP tips

It's meant as a training exercise.

iP: Grading

Total: 15 marks

Implementation [10 marks]: Required

- Achieve **more than 90% of all deliverables** by the end.
 - Requirements marked as **core** are required deliverables.
 - When a requirement specifies a minimum, the requirement is enough for it to be counted for grading -- however, we recommend you to go beyond the minimal; the farther you go, the more practice you will get.
- **The code quality** meets the following conditions:
 - Reasonable use of OOP e.g., at least some use of inheritance, code divided into classes in a sensible way
 - No blatant violations of the coding standard (both Java and Git conventions)
 - At least some errors are handled using exceptions
 - At least half of public methods/classes have javadoc comments
 - The code is neat e.g., no chunks of commented out code
 - Reasonable use of SLAP e.g., no very-long methods or deeply nested code
 - Has some JUnit tests

Project Management [2 marks]: To get full marks, you should achieve,

Objective: to help you level-up skills,
not to *differentiate* based on skill level
(i.e., **formative**, not summative)

No penalty for making mistakes, if you
learn from them, and rectify quickly.

Q₂

Answer this question in **Week 3 Quiz (on Canvas)**

Almost all are expected to score full marks for the iP.

- ☐ True
- ☐ False

Grading is more like S/U.

Some iP tips

Longer-term value beyond marks:

- Practice skills, learn tools (for the tP, and beyond)
 - weak programmer? catch up
 - strong programmer? go deeper, help others
- Use it to strengthen your resume
 - Make the product your own
 - Make it stand out



iP is a full product!

Q₃

In the iP, you are encouraged to follow the given command input/output formats closely.

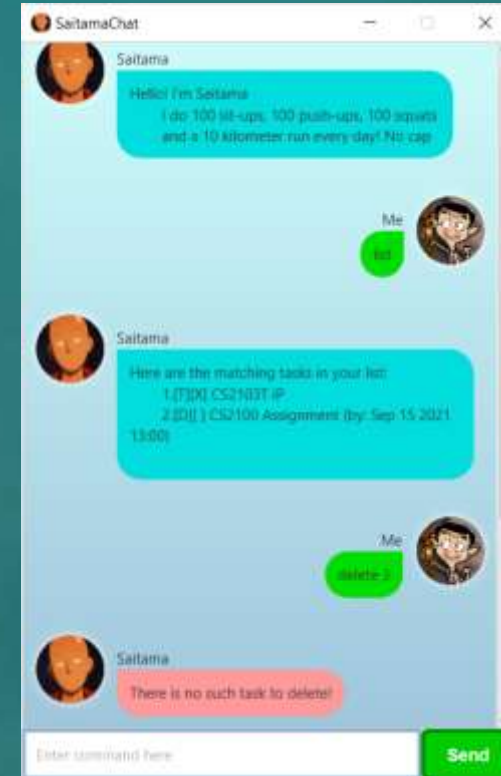
- ☐ True
- ☐ False

Q₄

Recommended: Come up with a *good* design that can support all iP increments indicated for future weeks.

- True
- False

list
Here are the tasks in your list:
1. ☒ read book
2. ☐ return book
3. ☐ buy bread
mark 2
Nice! I've marked this task as done:
☒ return book

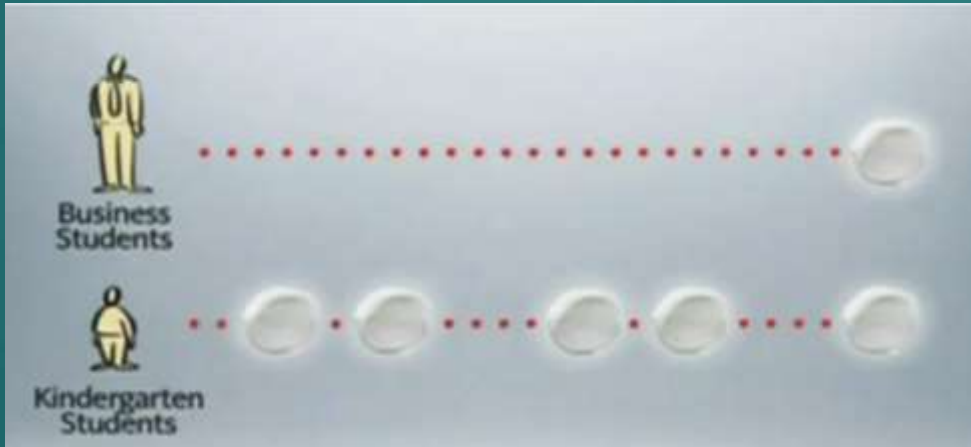


Don't forget to **go breadth-first iterative**: assume the future is unknown

(bonus: more practice + more experience → higher competence)



Always keep the marshmallow on the top!

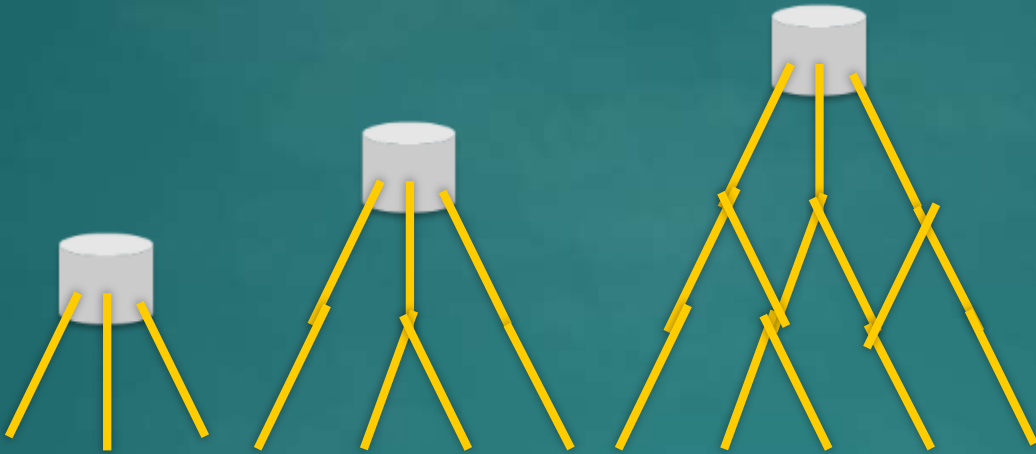


Solving small scale algorithm problems:

1. **Analyze** the problem
 2. **Design** a solution (as pseudo code?, diagram?)
 3. **Verify** the solution for correctness
 4. **Compare** with alternatives
 5. Sure of the solution? Now, **write the code**.
-

Creating bigger software:

1. Start with the **smallest code that works**
5 lines? Just an empty main method?
2. **Do a small change** to move it towards end goal.
Verify it still works. 5 more lines?
3. Keep repeating the above step.
4. Took a few wrong steps? Just undo them.
5. When thing get messy,
clean up, in small steps.



Past week: recap

Topics:

- [W2.1] SE: Intro ⚡⚡⚡
- [W2.2] SDLC Process Models: Basics ⚡⚡
- [W2.3] RCS: Revision History ⚡
- [W2.4] RCS: Remote Repos ⚡
- [W2.5] IDEs: Basic Features ⚡
- [W2.6] Automated Testing of Text UIs ⚡⚡

Admin:

1. Submit post-lecture quiz ⏰ Fri. P
2. Get connected to our communication channels

iP:

1. Learn about the project
2. Set up prerequisites
3. Set up the project in your computer
4. Add Increments while committing frequently:

Level-1 , Level-2 , Level-3 , Level-4 , A-
TextUiTesting , Level-5 , Level-6 , A-Enums

nus-cs2103- / forum

Type to search

Code Issues 8 Pull requests Discussions Actions Projects Wiki Security Insights Settings

Filters is:issue is:open Labels 11 Milestones 0

8 Open 72 Closed

Author Label Projects Milestones Assignee Sort

- Having an error with Automated Text UI Testing suitableForPeerHelp type: HelpRequest 4
- I can't find my tags on my repo suitableForPeerHelp type: HelpRequest 3
- Testing template code to help make testing easier type: Tip 2

Bonus participation
marks for forum
activities

Lecture briefing structure

Recap of the last week 

+

Preview of the next week

One more course briefing video in this week ...

- ✓ 1. Welcome
- ✓ 2. Goals
- ✓ 3. Iterative + Brownfield = ???
- ✓ 4. Challenges
- ➡ 5. Pitfalls
- 6. tP (parts: 1, 2a, 2b)
- 7. Topics Recap
- 8. Exam

[MUST-WATCH] Course Briefing Videos

One course briefing video to watch this week, given below:

CS2103/T Pitfalls (and how to avoid them)

➤ **Video** 9 mins

The weekly briefing for this week will be done in hybrid mode -- you can attend it F2F (@UTown-AUD2 Fri, Jan 24th from 4pm), join via Zoom, watch the recording later, or skip it altogether).

iP Help Session
the iP due to

Pitfalls
... and how to avoid them

Next week: overview

Topics:

- [W3.1] RCS: Branching ⚡⚡
- [W3.2] RCS: Creating Pull Requests ⚡
- [W3.3] Automating the Build Process ⚡
- [W3.4] Java: JavaDoc, file I/O, packages, JARs ⚡⚡
- [W3.5] Code Quality: Coding Standards ⚡⚡
- [W3.6] Developer Testing ⚡⚡
- [W3.7] Unit Testing ⚡

Admin:

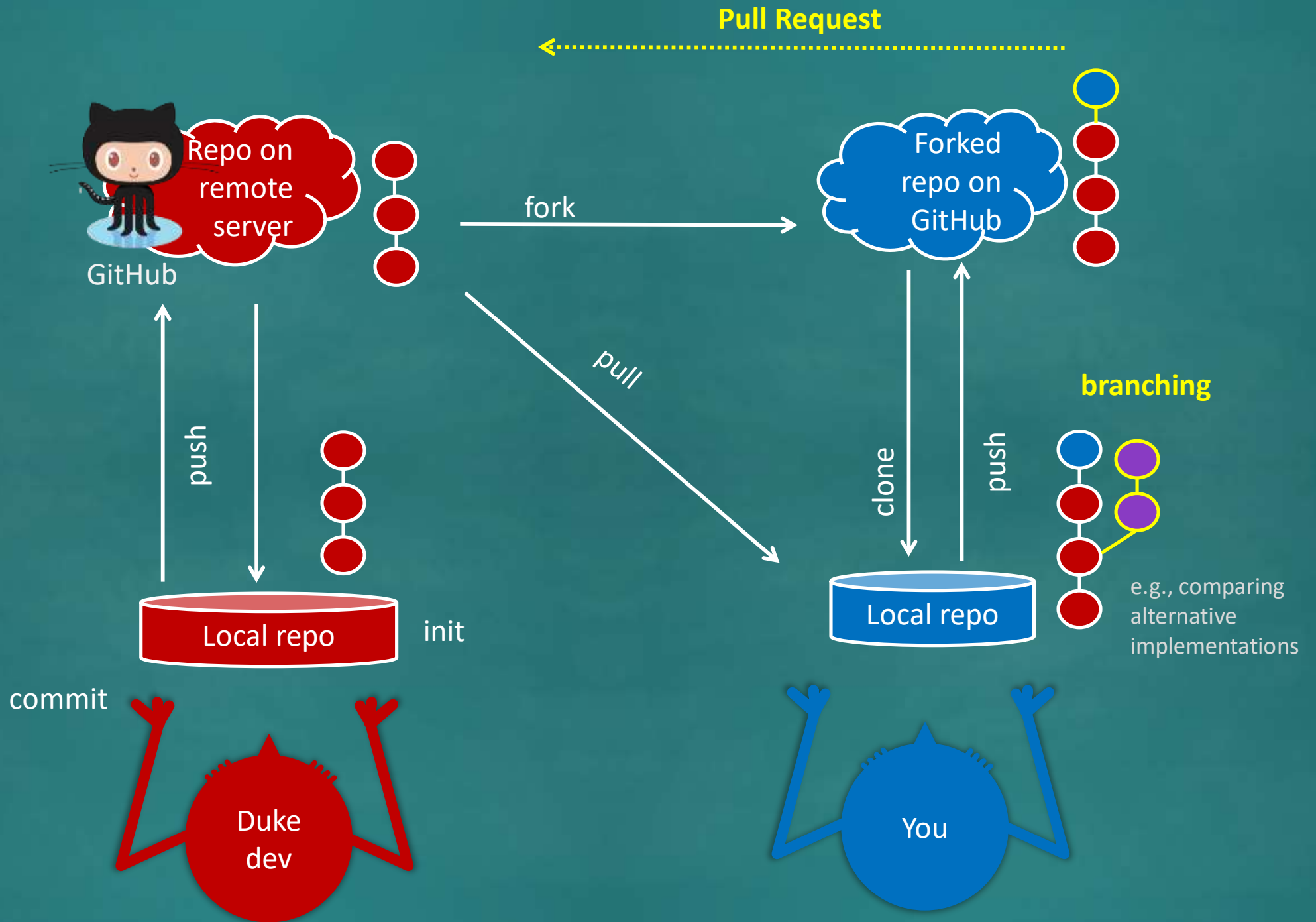
1. Submit weekly quiz 
2. [CS2103 students only] Form teams during the tutorial

iP:

1. Do any leftover iP tasks from the previous week
2. Create a PR to the upstream repo
3. Add Increments as branches: `Level-7`, `Level-8`
4. Add Increments: `A-MoreOOP`, `A-Packages`, `A-Gradle`, `A-JUnit`, `A-Jar`
5. Add Increments as parallel branches: `A-JavaDoc`, `A-CodingStandard`, `Level-9`
6. [Optional] Get a head start on JavaFX

tP:

1.  Individual Task: Explore AB3 features
 Sat 23:59
2.  Team Task: Set up a project meeting time
 by the end of the tutorial
3.  Team Task: Check up on team's collective iP status
 Tue, 



Next week: overview

Topics:

- [W3.1] RCS: Branching ⚡⚡
- [W3.2] RCS: Creating Pull Requests ⚡
- [W3.3] Automating the Build Process ⚡
- [W3.4] Java: JavaDoc, file I/O, packages, JARs ⚡⚡
- [W3.5] Code Quality: Coding Standards ⚡⚡
- [W3.6] Developer Testing ⚡⚡
- [W3.7] Unit Testing ⚡

Admin:

1. Submit weekly quiz 
2. [CS2103 students only] Form teams during the tutorial

iP:

1. Do any leftover iP tasks from the previous week
2. Create a PR to the upstream repo
3. Add Increments as branches: `Level-7`, `Level-8`
4. Add Increments: `A-MoreOOP`, `A-Packages`, `A-Gradle`, `A-JUnit`, `A-Jar`
5. Add Increments as parallel branches: `A-JavaDoc`, `A-CodingStandard`, `Level-9`
6. [Optional] Get a head start on JavaFX

tP:

1.  Individual Task: Explore AB3 features
 Sat 23:59
2.  Team Task: Set up a project meeting time
 by the end of the tutorial
3.  Team Task: Check up on team's collective iP status
 Tue,

Next week: overview

Topics:

- [W3.1] RCS: Branching ⚡⚡
- [W3.2] RCS: Creating Pull Requests ⚡
- [W3.3] Automating the Build Process ⚡
- [W3.4] Java: JavaDoc, file I/O, packages, JARs ⚡⚡
- [W3.5] Code Quality: Coding Standards ⚡⚡
- [W3.6] Developer Testing ⚡⚡
- [W3.7] Unit Testing ⚡

Admin:

1. Submit weekly quiz 
2. [CS2103 students only] Form teams during the tutorial

iP:

1. Do any leftover iP tasks from the previous week
2. Create a PR to the upstream repo
3. Add Increments as branches: `Level-7`, `Level-8`
4. Add Increments: `A-MoreOOP`, `A-Packages`, `A-Gradle`, `A-JUnit`, `A-Jar`
5. Add Increments as parallel branches: `A-JavaDoc`, `A-CodingStandard`, `Level-9`
6. [Optional] Get a head start on JavaFX

tP:

1.  Individual Task: Explore AB3 features
 Sat 23:59
2.  Team Task: Set up a project meeting time
 by the end of the tutorial
3.  Team Task: Check up on team's collective iP status
 Tue, 

Next week: overview

Topics:

- [W3.1] RCS: Branching ⚡⚡
- [W3.2] RCS: Creating Pull Requests ⚡
- [W3.3] Automating the Build Process ⚡
- [W3.4] Java: JavaDoc, file I/O, packages, JARs ⚡⚡
- [W3.5] Code Quality: Coding Standards ⚡⚡
- [W3.6] Developer Testing ⚡⚡
- [W3.7] Unit Testing ⚡

Admin:

1. Submit weekly quiz 
2. [CS2103 students only] Form teams during the tutorial

iP:

1. Do any leftover iP tasks from the previous week
2. Create a PR to the upstream repo
3. Add Increments as branches: `Level-7`, `Level-8`
4. Add Increments: `A-MoreOOP`, `A-Packages`, `A-Gradle`, `A-JUnit`, `A-Jar`
5. Add Increments as parallel branches: `A-JavaDoc`, `A-CodingStandard`, `Level-9`
6. [Optional] Get a head start on JavaFX

tP:

1.  Individual Task: Explore AB3 features
 Sat 23:59
2.  Team Task: Set up a project meeting time
 by the end of the tutorial
3.  Team Task: Check up on team's collective iP status
 Tue,

An example commit message

Stated in the
imperative mood

Update Developer Guide from Addressbook-Level1

Commit message subject
is one of the first things
someone will notice
when you submit code

When
merged,
this
commit will

Update Developer Guide to use Java 17

--- [blank line] ----

The Developer Guide instructs users to use Java 11 or later. The code is already compatible with Java 17.

Since AddressBookL4 is already using Java 17, this project has become inconsistent with AB-4.

Let's update the Developer Guide to mention using Java 17 or later.

Subject

Current state

What's wrong with
the current state

How this commit
changes the current
state

Other relevant info
e.g., references

Admin info

Module Overview

Using this Website ⚡1

Module Expectations ⚡2

Module Structure ▾

Instructors ⚡4

Textbooks ⚡2

Programming Language ⚡2

Standards/Conventions ⚡2

Individual Project (iP) ▾

Team Project (tP) ▾

Peer Evaluations ⚡2

Tools ⚡3

⊕ Programming Language

Standards/Conventions

Given below are the standards and conventions to follow



When something is not covered by the given
style you want to follow for that, but try to be
everyone in the team follows the same (if it is a t

Java

- **REQUIRED** Follow the *basic and intermediate rules* in
- **OPTIONAL** Follow *advanced rules*.

Git

- **REQUIRED** Follow the Git *commit message subject conventions* in the [SE-EDU Git conventions](#).
- **OPTIONAL** Follow conventions for the commit message body.

Markdown

- **OPTIONAL** Follow [Markdown style guide at SE-EDU](#).

Documentation

Report Bugs

Forum

Gitter (Chat)

Instructors

Announcements

Files

Tutorial Schedule

Java Coding Standard

Git Conventions

Forum Activities Dashboard

Participation Dashboard

al Project): Overview ⊕

to choose which
(any), and ensure

Must-do in the iP:

- ☐ Follow the given Git convention for future commit message **subject**
- ☐ Follow the given Git convention for future commit message **body**
- ☐ Modify **past commits** to comply with the given Git conventions

Yes, it is possible to change past commits.

Pushing edited commits might require a **force-push**.

Not recommended if you have pushed those commits to a remote repo already, and others have pulled them.







TEAMMATES

1.1 million users

150,000 Lines of Code



500+ student developers

Java EE - CityConnect-TDD/CityConnect-TDD/src/CityConnectTest.java - Eclipse

File Edit Source Refactor Navigate Search Project CodePro Run Window Help

Quick Access Java EE Debug Java Monitor

Project Explorer Results of runni...

Tests: 0/1 Methods: 243

Search: Passed: 243 Failed: 0 Skipped: 0

All Tests Failed Tests Summary

Failure Exception

Markers Console Progress Search Coverage JUnit

Component tests [TestNG] C:\Program Files\Java\jdk1.7.0_03\bin\javaw.exe (Sep 5, 2013 11:45:56 AM)

- * non-registered users cannot access
- * students cannot access
- * other course instructor cannot access
- * course instructor can access
- * admin can access

[=====teammates.test.cases.ui.InstructorEvalAddActionTest=====]

- * not-logged-in users cannot access
- * non-registered users cannot access
- * students cannot access
- * other course instructor cannot access
- * course instructor can access
- * admin can access

[=====teammates.test.cases.ui.InstructorEvalsPageActionTest=====]

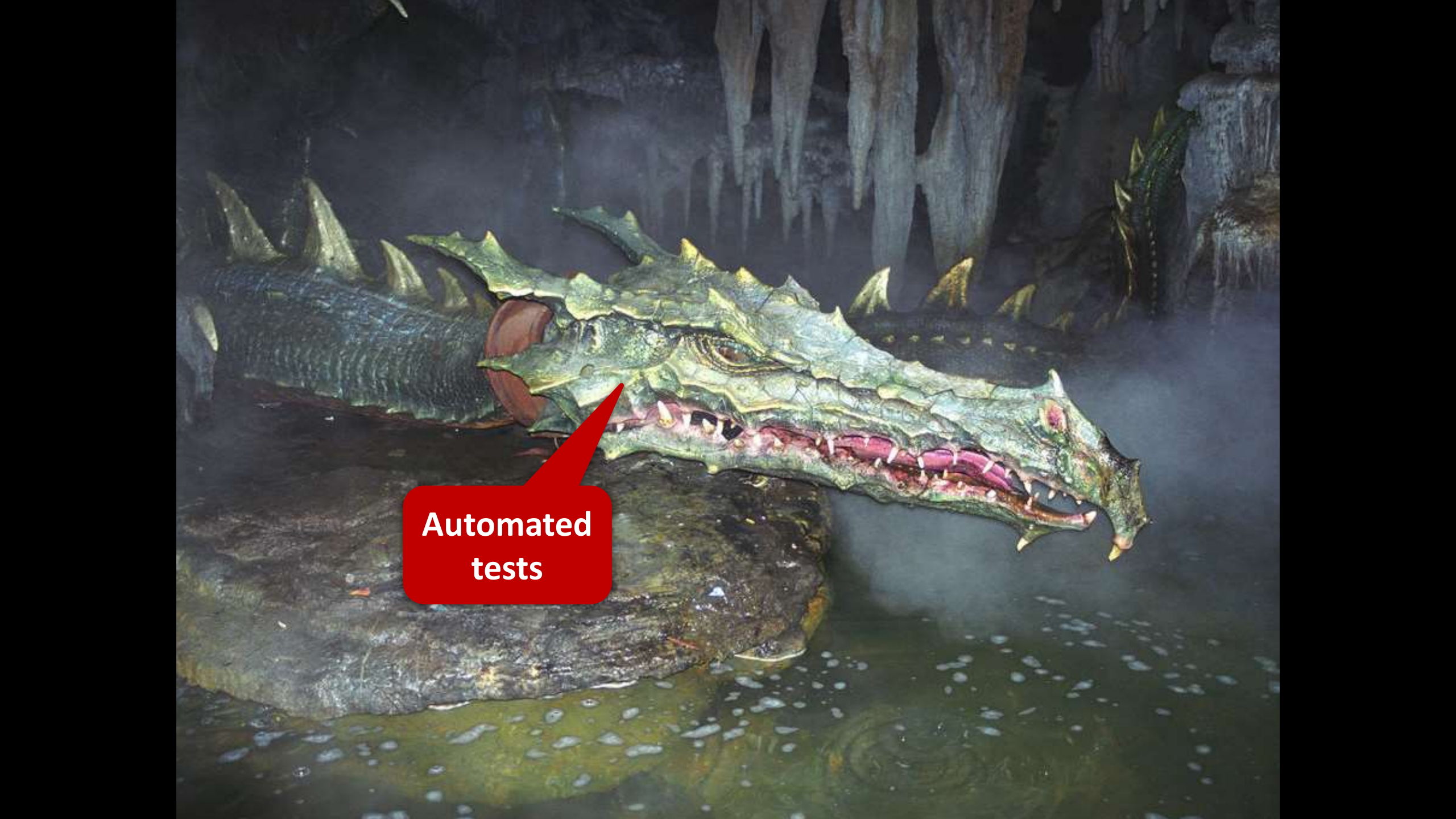
- * not-logged-in users cannot access
- * non-registered users cannot access
- * students cannot access
- * course instructor can access
- * admin can access
- * Typical case, 2 courses
- * Masquerade mode, 0 evaluations
- * Masquerade mode, 0 courses

[=====teammates.test.cases.ui.InstructorEvalDeleteActionTest=====]

- * not-logged-in users cannot access
- * non-registered users cannot access
- * students cannot access
- * other course instructor cannot access
- * course instructor can access
- * admin can access

[=====teammates.test.cases.ui.InstructorEvalEditPageActionTest=====]

- * not-logged-in users cannot access
- * non-registered users cannot access
- * students cannot access
- * other course instructor cannot access
- * course instructor can access
- * admin can access

A large, detailed sculpture of a dragon's head, primarily green with yellow and gold accents on its scales and spikes. The dragon's mouth is open, revealing sharp white teeth and a pinkish-red interior. It is positioned in a dark, cave-like environment with stalactites hanging from the ceiling and a body of water in the foreground. A red speech bubble points to the dragon's mouth.

**Automated
tests**

Next week: overview

Topics:

- [W3.1] RCS: Branching ⚡⚡
- [W3.2] RCS: Creating Pull Requests ⚡
- [W3.3] Automating the Build Process ⚡
- [W3.4] Java: JavaDoc, file I/O, packages, JARs ⚡⚡
- [W3.5] Code Quality: Coding Standards ⚡⚡
- [W3.6] Developer Testing ⚡⚡
- [W3.7] Unit Testing ⚡

Admin:

1. Submit weekly quiz 
2. [CS2103 students only] Form teams during the tutorial

iP:

1. Do any leftover iP tasks from the previous week
2. Create a PR to the upstream repo
3. Add Increments as branches: `Level-7`, `Level-8`
4. Add Increments: `A-MoreOOP`, `A-Packages`, `A-Gradle`, `A-JUnit`, `A-Jar`
5. Add Increments as parallel branches: `A-JavaDoc`, `A-CodingStandard`, `Level-9`
6. [Optional] Get a head start on JavaFX

tP:

1.  Individual Task: Explore AB3 features
 Sat 23:59
2.  Team Task: Set up a project meeting time
 by the end of the tutorial
3.  Team Task: Check up on team's collective iP status
 Tue, 

Next week: overview

Topics:

- [W3.1] RCS: Branching ⚡⚡
- [W3.2] RCS: Creating Pull Requests ⚡
- [W3.3] Automating the Build Process ⚡
- [W3.4] Java: JavaDoc, file I/O, packages, JARs ⚡⚡
- [W3.5] Code Quality: Coding Standards ⚡⚡
- [W3.6] Developer Testing ⚡⚡
- [W3.7] Unit Testing ⚡

Serves as an anchor point to stabilize the rhythm of the iterations

Admin:

1. Submit weekly quiz **P**
2. [CS2103 students only] Form teams during the tutorial

iP:

1. Do any leftover iP tasks from the previous week
2. Create a PR to the upstream repo
3. Add Increments as branches: **Level-7**, **Level-8**
4. Add Increments: **A-MoreOOP**, **A-Packages**, **A-Gradle**, **A-JUnit**, **A-Jar**
5. Add Increments as parallel branches: **A-JavaDoc**, **A-CodingStandard**, **Level-9**
6. [Optional] Get a head start on JavaFX

tP:

1. Individual Task: Explore AB3 features
🕒 Sat 23:59
2. Team Task: Set up a project meeting time
🕒 by the end of the tutorial
3. Team Task: Check up on team's collective iP status
🕒 Tue,

No need to decide on a project yet though.

Some topic videos have in-video quizzes

▼

W3.5b

Quality Assurance → Testing → Developer Testing → Why

>

Video

Q+

🏆 Can explain the need for early developer testing

Delaying testing until the full product is complete has a number of disadvantages:

- **Locating the cause of a test case failure is difficult due to a large search space;** in a large system, the search space could be millions of lines of code, written by hundreds of developers! The failure may also be due to multiple inter-related bugs.

Tutorial mode

First 2 weeks: fully online (Zoom).

- 2 tutors (10 students each) in each tutorial slot.
- Everyone is expected to answer questions posed by the tutor (more about this policy in next week).

Moving to F2F later. More info to come later.

iP Help Session

- Anybody here to help others?
- Anybody needs Mac-specific help?
- Anybody else seeking help?